1 Evaluation method

In line with the specification, my aim in this section was to evaluate how changes to the grid size (N) or the number of colours (D) affect the path cost and the number of moves. I created two algorithms (`src/eval/EvalD.py` and `src/eval/EvalN.py`) that randomly generate problem sets in which the grid size or the number of colours (respectively) are gradually increased. The data presented in this document are drawn from `D_results.csv` and `N_results.csv`, which were generated on the lab computers using the evaluation scripts.

It should be noted that this method of generating test data is not perfect. The random generation means that not all grids will have a solution, nor will there be consistency in the problem set or the data generated between runs of the evaluation script. If I had more time, I would have implemented my own problem sets so that greater insights could have been acquired.

2 Observations when varying the grid size (N)

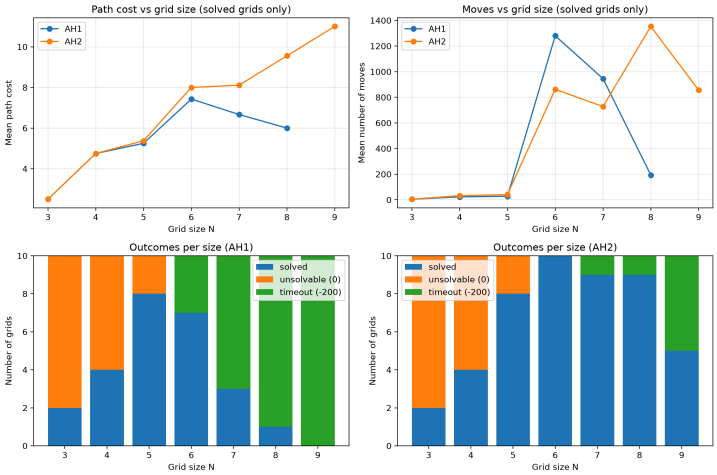


Figure 1: Shows 4 graphs. The top-left graph is a plot of the mean path cost against the grid size. The top-right graph plots the mean number of moves against the grid size. These top graphs only evaluate the cases where a solution was found. The bottom two graphs are bar charts (Number of grids against grid size) demonstrating the distribution of solved, unsolved and timed-out grid configurations per algorithm

Between `N = 3` and `N = 5`, AH1 and AH2 keep relatively the same path cost and number of moves. This is reflected in the bar charts, where AH1 and AH2 gain the same number of results for these N values without timing out. By `N = 6`, the AH1 algorithm appears to suddenly plateau: shallowly for path cost, but steeply for number of moves. It could be mistakenly assumed that AH1 is therefore more efficient; instead, the bar chart on the bottom-left reveals that from `N = 6` onwards, AH1 times out at an increasing rate. This results in the mean being calculated using a significantly smaller data set than that of its previous N values and hence skewing the data in a particular direction that could have been avoided with a fuller data set. The timing-out behaviour of AH1 is a result of its heuristic calculation. Since its heuristic is the number of colours remaining, for these particular tests its heuristic remains as one of three values (1, 2 or 3). Therefore, as the grid size increases, the number of repeat h-value frontier nodes also increases, leading to more explorations being required before it can find something that works. By comparison, AH2 is still able to have more variations in its heuristic value. This is reflected in AH2's ability to solve grids for every `N`, with significantly fewer timeouts.

3 Observations when varying the number of colours (D)

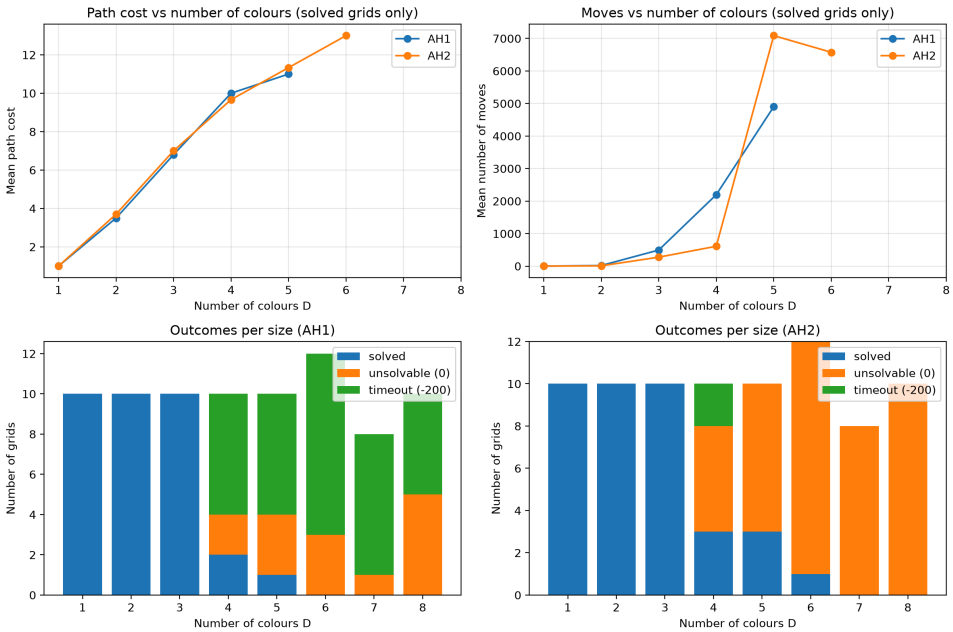


Figure 2: Shows 4 graphs. The top-left graph is a plot of the mean path cost against the number of colours. The top-right graph plots the mean number of moves against the number of colours. These top graphs only evaluate the cases where a solution was found. The bottom two graphs are bar charts (Number of grids against number of colours) demonstrating the distribution of solved, unsolved and timed-out grid configurations per algorithm

For this case, I chose to use a fixed-size grid of size 6 and a minimum of 1 colour up to a maximum of 8 colours. The size was chosen because it has been proven largely solvable for both AH1 and AH2 in the previous section and has enough grid squares that increasing the number of colours wouldn't impact the grid's ability to have a solution too greatly.

Interestingly, in the top-left plot, the average path costs of AH1 and AH2 remain almost identical throughout. In other words, when a solution is found, both algorithms are largely able to agree on the returned path cost. Furthermore, the top-right plot shows that AH1 remains in a steadily increasing curve - even for `D = 4` and `D = 5`, where the number of solved grids significantly drops. This deviates from its 'jumping' behaviour in the previous section. Furthermore, AH1 is able to complete significantly more moves overall in comparison to the previous section (a peak of 5000 moves here, compared to a peak of just over 1200 moves there). This is because AH1 is finally able to utilise its colour-based heuristic for solving. By comparison, AH2 still suffers from some jumping: it initially has a steady, shallow increase between `D = 1` and `D = 4` and then suddenly jumps to 7000 average moves by `D = 5`. One reason for this significant deviation lies within the data used to plot this point. For `D = 5` under AH2, there are only 3 cases where grids were able to be solved, and the moves each of these cases took are as follows: ['10147', '9530', '1584']. The combination of a smaller data set to calculate a mean from and the massive variation in the data will have caused this point to jump so significantly and reflects some of the drawbacks of a randomised problem set. That being said, this still highlights AH2's capability of attempting far more moves without timing out in comparison to AH1. In fact, AH2 only times out on two occasions.

4 Conclusions

When the problem complexity is smaller, AH1 and AH2 behave almost identically. However, as the complexity of the problem grows, the number of moves AH1 attempts increases significantly; thus it times out. Meanwhile, AH2 is still largely able to find solutions before reaching the point of a timeout.